



SecPass

Documentação Técnica Completa — Arquitetura, Segurança e Funcionamento

Gerenciador de senhas multiplataforma (Android, iOS, macOS e Windows)

Cortexis Tech

Documento atualizado em 10 de setembro de 2026

1. Visão Geral

SecPass é um gerenciador de senhas multiplataforma com dois clientes que compartilham a mesma lógica de cofre, criptografia e sincronização: um aplicativo **mobile** (Android e iOS, construído em React Native/Expo) e um aplicativo **desktop** para macOS e Windows (construído em Electron). Os clientes entendem o mesmo formato de cofre cifrado, o que permite que um usuário crie um item em um Mac e o veja, já decifrado, no celular ou no PC Windows — e vice-versa — através de um backend de sincronização à sua escolha.

O produto foi desenhado em torno de um princípio central: **criptografia de ponta a ponta (E2E) client-side**. O servidor de sincronização (Google Drive ou iCloud) nunca vê uma senha em texto claro — ele armazena apenas blobs cifrados. A chave de criptografia é derivada localmente, em cada dispositivo, a partir do e-mail e da senha de acesso do usuário, e nunca trafega pela rede.

1.1 Plataformas e propósito de cada uma

Cliente	Tecnologia	Uso típico
Android	React Native + Expo (SDK 57)	App principal, biometria via <code>expo-local-authentication</code>
iOS	React Native + Expo + módulo nativo Swift	Mesmo código React, mas com acesso nativo ao CloudKit via módulo Swift próprio
macOS	Electron (Chromium + Node.js)	App desktop independente, reaproveitando os módulos JS de criptografia/sync do mobile
Windows	Electron (mesmo código-fonte do macOS)	Mesmo app desktop, empacotado como instalador NSIS x64 — sem dependência de nenhuma API específica da Apple (iCloud não se aplica nesta plataforma)

1.2 Modelo de sincronização escolhido pelo usuário

Uma decisão de produto importante tomada durante o desenvolvimento: em vez de escolher automaticamente um backend de nuvem, o app deixa o **usuário decidir, por aparelho**, qual ecossistema de sincronização usar:

- **Google Drive** (pasta oculta `appDataFolder`, invisível ao usuário no Drive normal) — funciona entre Android, iOS, Mac e Windows. É a única opção disponível no Windows, e a recomendada para quem usa uma mistura de sistemas (ecossistema "híbrido").
- **iCloud (CloudKit)** — funciona apenas entre iPhone e Mac. É a opção recomendada para quem usa exclusivamente produtos Apple, evitando depender de uma conta Google.

Essa escolha é armazenada localmente por dispositivo (`syncPreference.js`) e pode ser trocada a qualquer momento pela tela de "Sincronização", com migração dos dados existentes para o novo backend.

2. Arquitetura Geral

O projeto é um monorepo (sem uso de submódulos/workspaces formais) onde a pasta raiz é o app mobile, e a pasta `desktop/` contém o app Electron (que roda tanto em macOS quanto em Windows a partir do mesmo código-fonte). O desktop importa e reaproveita diretamente vários arquivos de lógica de negócio do mobile (adaptados para rodar em Node.js puro, sem APIs do React Native).

SecPass (monorepo)



2.1 Por que o desktop usa CloudKit "Web Services" e não o framework nativo

A Apple não expõe o framework nativo `CloudKit` para aplicações que não sejam apps Swift/Objective-C empacotados como app Mac "de verdade" via Xcode com entitlements de iCloud — um app Electron não tem esse acesso. A solução adotada foi usar a biblioteca `cloudkit.js` da Apple (a mesma usada em páginas web que integram com iCloud), rodando dentro de uma `BrowserWindow` do Electron oculta (não visível ao usuário), controlada via uma ponte de IPC (processo principal $\rightleftharpoons$ janela oculta $\rightleftharpoons$ processo principal). Essa abordagem funciona, mas não é um cenário oficialmente documentado pela Apple para esse fim. **No Windows, essa opção simplesmente não existe** — o backend de sincronização disponível é exclusivamente Google Drive.

3. Estrutura de Pastas

3.1 App mobile (raiz do repositório)

Pasta/Arquivo	Conteúdo
<code>src/app/</code>	Rotas do Expo Router (entrada do app) — inclui uma rota "fantasma" <code>oauth2redirect.tsx</code> que existe só para absorver o deep link de retorno do OAuth do Google no Android sem mostrar tela de erro (ver 6.1.2)
<code>src/screens/HomeScreen.js</code>	Tela principal — login, cofre, listagem de itens (com pull-to-refresh), todos os modais (sincronização, histórico de segurança, etc.). Arquivo central do app, ~2.600 linhas.
<code>src/components/</code>	Componentes reutilizáveis: cartão de senha (<code>PasswordCard.js</code>), formulário (<code>PasswordForm.tsx</code>), busca, logo, créditos
<code>src/services/</code>	Toda a lógica de negócio: criptografia, storage, autenticação, auditoria (detalhado na seção 4 em diante)
<code>src/hooks/</code> , <code>src/utils/</code>	Hooks e utilitários de apoio
<code>modules/secure-vault-sync/</code>	Módulo nativo Expo (Swift) usado apenas no iOS para autenticação/token do Google Drive via SDK nativo
<code>modules/secure-vault-cloudkit/</code>	Módulo nativo Expo (Swift) que fala diretamente com o framework CloudKit nativo do iOS
<code>plugins/</code>	Config plugins customizados do Expo (ver seção 11)
<code>patches/</code>	Patches permanentes via <code>patch-package</code> (ver seção 11)
<code>__tests__/</code>	Suíte de testes Jest (210 testes, 17 arquivos)
<code>android/</code> , <code>ios/</code>	Projetos nativos gerados pelo <code>expo prebuild</code> (não editados manualmente — regeneráveis)

3.2 App desktop (`desktop/` — macOS e Windows)

Pasta/Arquivo	Conteúdo
<code>src/main/index.js</code>	Processo principal do Electron — cria janelas, registra handlers de IPC (inclui <code>vault:refresh</code> , o handler do botão "Atualizar")
<code>src/main/core/</code>	Portas Node.js dos mesmos serviços do mobile (criptografia, storage, sync, etc.)

<code>src/main/cloudkit/</code>	Janela oculta + ponte IPC para rodar CloudKit JS (ver seção 2.1) — só relevante em macOS
<code>src/main/secureStore.js</code>	Armazenamento seguro local usando o cofre de credenciais do sistema operacional (via <code>safeStorage</code> do Electron — Keychain no macOS, DPAPI no Windows)
<code>src/preload/</code>	Scripts de preload — ponte segura e controlada entre o processo principal e o renderer
<code>src/renderer/</code>	Interface React do app desktop (botão "Atualizar" no toolbar, ver seção 6.5)
<code>build/</code>	Ícones de marca do executável (<code>icon.icns</code> para macOS, <code>icon.ico</code> para Windows, <code>icon.png</code> como fonte) e <code>entitlements.mac.plist</code> para assinatura com hardened runtime
<code>__tests__/</code>	Suíte de testes Jest própria (64 testes, 6 arquivos)

4. O Cofre e o Modelo de Criptografia

4.1 Derivação de chave

Quando o usuário cria ou desbloqueia uma conta local, a senha de acesso e o e-mail são usados para derivar as chaves de criptografia via **PBKDF2-SHA256 com 600.000 iterações** — um custo computacional deliberadamente alto para dificultar ataques de força bruta offline caso um atacante obtenha uma cópia do cofre cifrado. Dessa derivação nascem várias sub-chaves com propósitos distintos (chave de cifragem, chave de HMAC/verificação, chave de selo do log de auditoria), cada uma derivada de forma independente (via HMAC) para que o comprometimento de uma não comprometa as outras.

4.2 Cifragem dos itens (envelope)

Cada item do cofre (uma credencial: site, usuário, senha, notas) é cifrado individualmente com **AES-256-GCM** — uma cifra autenticada (AEAD), que garante não só sigilo como também integridade: qualquer alteração de um único byte do texto cifrado é detectada na decifragem. O formato do envelope evoluiu em três versões ao longo do projeto:

Versão	Cifra	Observação
v1 (legado)	AES-256-CBC + HMAC-SHA256 separado	Formato antigo, mantido só para conseguir ler cofres muito antigos
v2	AES-256-GCM	Passou a usar uma cifra autenticada única (AEAD) em vez de CBC+HMAC
v3 (atual)	AES-256-GCM com AAD vinculada	Além da cifragem, vincula <code>id</code> , data de revisão (<code>updatedAt</code>) e a flag de exclusão (<code>tombstone</code>) como dados associados autenticados (AAD) do AES-GCM

Por que a v3 existe: nas versões anteriores, um item cifrado válido podia, em tese, ser "reaproveitado" por alguém com acesso de escrita ao backend remoto — substituindo um item atual por uma cópia antiga (porém genuína, cifrada com a mesma senha), sem que a decifragem detectasse nada de errado. Ao vincular a data de revisão e o estado de exclusão na AAD, qualquer tentativa de "voltar no tempo" um item faz a verificação do AEAD falhar, e o item é rejeitado na decifragem.

4.3 Formato do cofre completo

```
{
  id: string,           // identificador unico do item
  envelope: {...},      // dados cifrados (ver 4.2)
  updatedAt: number,    // timestamp da ultima revisao (texto claro)
```

```
tombstone: boolean // true = item excluído (soft-delete, sincroniza exclusões)
}
```

Note que `updatedAt` e `tombstone` ficam em texto claro no registro (não dentro do envelope cifrado) — isso é necessário para que o algoritmo de merge (seção 4.4) e a sincronização consigam decidir qual versão de um item é mais recente *sem precisar decifrar nada*.

4.4 Sincronização e merge multi-dispositivo

Como não existe um servidor central que arbitra conflitos, o merge de itens entre dispositivos segue uma estratégia **last-write-wins por item**, comparando o campo `updatedAt` de cada lado (local vs. remoto) e mantendo o mais recente, implementada em `vaultMerge.js`. Itens excluídos em um dispositivo se propagam como "tombstones" para os demais, em vez de serem removidos silenciosamente — isso evita que um dispositivo offline "ressuscite" um item que foi apagado em outro lugar.

5. Autenticação e Controle de Acesso

5.1 Conta local e senha de acesso

O SecPass não tem um "servidor de contas" — a "conta" é inteiramente local a cada dispositivo. A mesma combinação de email+senha em dois dispositivos diferentes gera a mesma chave de derivação, o que permite a sincronização funcionar sem nunca enviar a senha para lugar nenhum.

5.2 Biometria

No mobile, o desbloqueio do cofre pode usar biometria (Face ID/Touch ID/impressão digital) via `expo-local-authentication`. No desktop, o mesmo padrão é usado com Touch ID no Mac quando disponível (no Windows, o desbloqueio é sempre por senha, já que não há um equivalente universal ao Touch ID nesta plataforma).

5.3 Proteção contra força bruta local

Tentativas de senha incorretas incrementam um contador persistido em disco (`loginGuard.js`), que aciona bloqueios progressivos com tempo de espera crescente após múltiplas falhas. Esse estado é salvo **antes** de atualizar a interface.

5.4 Proteção contra captura de tela

No mobile, a tela é protegida contra captura (screenshot) e contra exibição de conteúdo sensível no "app switcher" do sistema (via `expo-screen-capture`).

5.5 Limpeza automática da área de transferência

Quando o usuário copia uma senha, o valor é apagado automaticamente após 30 segundos, tanto no mobile quanto no desktop.

6. Sincronização Entre Dispositivos

6.1 Google Drive

Usa a `appDataFolder` da Google Drive API — uma pasta especial, invisível na interface normal do Drive, acessível apenas pelo próprio app. A forma de autenticação difere por plataforma:

- **iOS:** SDK nativo `@react-native-google-signin/google-signin` (fluxo OAuth do próprio sistema). Continua funcionando normalmente.
- **Android:** fluxo OAuth 2.0 + PKCE via navegador do sistema (`expo-web-browser`), **não** o SDK nativo. Ver justificativa detalhada em 6.1.1.
- **Desktop (macOS e Windows):** fluxo OAuth "para aplicativo de computador" via servidor HTTP loopback local + PKCE, idêntico em ambos os sistemas operacionais.

6.1.1 Por que o Android não usa mais o SDK nativo do Google Sign-In

O SDK nativo (`GoogleSignin.signIn` + `requestScopes`) apresentava uma falha consistente e reproduzida em **quatro certificados de assinatura diferentes** (keystore gerenciado pelo EAS, keystore de release local, keystore de debug local, keystore de debug do emulador), em **dois aparelhos físicos diferentes mais um emulador**, sempre ao solicitar o escopo sensível `drive.appdata`: o erro `AutoManageHelper: Unresolved error while connecting client` no logcat, sem mensagem mais específica. O mesmo SDK, com o mesmo escopo, funciona normalmente no iOS.

A investigação descartou incompatibilidade de certificado (testado exaustivamente) e apontou para um padrão relatado por terceiros: escopos OAuth sensíveis via SDK nativo do Google no Android podem exigir que o app esteja distribuído pela Play Store para o Google conceder o consentimento — algo não viável de testar/depender neste projeto (sem conta de desenvolvedor Play Console configurada). A solução adotada foi trocar a abordagem inteira no Android: um fluxo OAuth via navegador (mesmo princípio já usado com sucesso pelo app desktop), reaproveitando o client OAuth do tipo "iOS" já existente (não um client "Android" novo), já que esse fluxo não depende de validação por pacote+SHA-1. Um config plugin (`plugins/withAndroidGoogleOAuthRedirect.js`) registra o intent-filter necessário no `AndroidManifest.xml` gerado para capturar o retorno do navegador.

6.1.2 Colisão entre o retorno do OAuth e o roteamento do Expo Router (Android)

Durante o teste em build de produção real (minificado, R8/ProGuard) em aparelho físico, foi identificado que o retorno do navegador após o login do Google era capturado corretamente pelo `expo-web-browser` (o token era obtido e salvo com sucesso), mas o Expo Router — que escuta os mesmos eventos de deep link via `Linking` do React Native para seu próprio sistema de rotas — também recebia esse mesmo evento e tentava navegar para ele como se fosse uma tela do app, exibindo sua página padrão de "rota não encontrada" por cima da interface, mesmo com o login já tendo sido concluído nos bastidores. A correção foi registrar uma rota "fantasma"

(`src/app/oauth2redirect.tsx`) que existe apenas para dar ao Router uma correspondência válida e redirecionar imediatamente de volta à tela inicial, sem exibir nenhum conteúdo visível.

6.2 iCloud (CloudKit)

No iOS, a integração é nativa, via o módulo Swift `secure-vault-cloudkit`. No desktop, apenas em macOS (ver seção 2.1) — não disponível no Windows.

6.3 Escolha manual e migração de backend

A preferência de backend (`syncPreference.js`) é armazenada por dispositivo, não por conta. A troca de backend roda uma migração que envia os itens do cofre atualmente decifrados na memória para o novo backend antes de trocar a preferência.

6.4 Confirmação ao encontrar um cofre remoto já existente (dispositivo novo)

Quando alguém instala o app pela primeira vez num dispositivo novo e informa um email/senha que já tem um cofre remoto associado, o app mostra um resumo desse cofre — quantidade de itens e data da última modificação — e pede confirmação explícita antes de baixá-lo e decifrá-lo.

6.5 Atualização manual (pull-to-refresh e botão "Atualizar")

O pull remoto automático acontece ao desbloquear o cofre (após o app voltar de segundo plano, já que sair do app bloqueia o cofre por segurança — ver 5.4). Isso deixava um caso descoberto: o app aberto e desbloqueado em primeiro plano, enquanto outro dispositivo grava uma mudança no backend remoto, só seria percebido no próximo ciclo de bloqueio/desbloqueio. Para cobrir esse caso, foram adicionados:

- **Mobile:** gesto de "puxar para atualizar" (pull-to-refresh) na lista principal de credenciais, visível apenas quando a sincronização com o Drive está ativa.
- **Desktop:** botão de "Atualizar" na barra de ferramentas (ícone de seta circular), que chama o handler IPC `vault:refresh` — re-busca o backend remoto e faz o merge com o estado atual em memória, sem precisar bloquear/desbloquear o cofre.

7. Auditoria e Segurança Adicional

7.1 Log de auditoria de segurança com selo à prova de adulteração

O app mantém um log local de eventos de segurança, protegido por uma **cadeia de hashes HMAC**: cada novo lote de eventos gera um hash encadeado (SHA-256), assinado com uma chave HMAC derivada da própria senha do cofre (`deriveSealKey`), então só quem sabe a senha consegue gerar um selo válido.

7.2 Monitoramento de erros com redação de dados sensíveis

O app usa o Sentry para monitoramento de erros em produção. Antes de qualquer evento ser enviado, um filtro de redação (`errorMonitoring.ts`) remove ou mascara padrões que possam conter dados sensíveis.

7.3 Segurança específica do desktop

Mecanismo	Descrição
Armazenamento seguro local	<code>secureStore.js</code> usa o <code>safeStorage</code> do Electron (Keychain no macOS, DPAPI no Windows). Se a criptografia do sistema operacional não estiver disponível, o app recusa salvar em texto claro em vez de degradar silenciosamente a segurança.
Isolamento de processos (sandbox)	As janelas do Electron rodam com o sandbox padrão ativado.
Janela oculta do CloudKit	Restringe navegação apenas aos domínios da Apple necessários — só relevante em macOS.
Validação de entrada	Campos de credenciais são validados antes de persistir no disco.

8. Módulos Nativos (Swift, iOS)

Dois módulos nativos escritos em Swift, empacotados como Expo Config Modules, dão ao app iOS acesso a funcionalidades que não existem em bibliotecas JavaScript genéricas:

Módulo	Função
<code>secure-vault-sync</code>	Gerencia autenticação e obtenção de tokens para o Google Drive de forma nativa no iOS.
<code>secure-vault-cloudkit</code>	Fala diretamente com o framework CloudKit nativo da Apple.

O Android **não** tem (nem precisa mais de) um módulo nativo equivalente para autenticação com o Drive: o fluxo via navegador (seção 6.1.1) usa apenas bibliotecas JS (`expo-web-browser` , `react-native-quick-crypto` para PKCE, `expo-secure-store` para persistência do refresh token).

9. Auditoria de Segurança Realizada Nesta Fase do Projeto

O projeto passou por uma revisão de segurança adversarial completa, na qual foram identificadas e corrigidas 11 vulnerabilidades/lacunas, resumidas abaixo:

#	Achado	Correção
1	Reaproveitamento/rollback de envelopes antigos via backend comprometido	AAD vinculando <code>updatedAt</code> / <code>tombstone</code> ao AES-GCM (envelope v3, seção 4.2)
2	Um item corrompido derrubava a decifragem do cofre inteiro	Decifragem isolada por item, com try/catch individual
3	Bloqueio de login podia ser burlado matando o app no meio de uma tentativa	Persistência do estado de bloqueio antes de atualizar a UI
4	Tela de login ficava sem proteção contra captura de tela	Proteção movida para ativação incondicional, desde o boot da tela
5	Log de auditoria podia ser adulterado sem detecção	Selo HMAC encadeado (seção 7.1)
6	Cofre remoto de dispositivo novo era aceito sem nenhuma verificação humana	Resumo + confirmação explícita antes de decifrar (seção 6.4)
7	Desktop podia gravar segredos em texto claro se a criptografia do SO estivesse indisponível	<code>secureStore</code> agora recusa a operação em vez de degradar
8	Desktop não validava campos de credencial antes de persistir	Validação adicionada antes de toda escrita
9	Sandbox do Electron estava desabilitado em algumas janelas	Sandbox reativado (configuração padrão, mais segura)
10	Área de transferência do desktop não limpava a senha copiada	Auto-limpeza após 30s (igual ao mobile)
11	Regex de redação do Sentry tinha lacunas; exceção de rede local desnecessária no iOS	Regex ampliado; <code>NSAllowsLocalNetworking</code> desativado

Após a adoção do cliente Windows e a troca da autenticação do Android, foi conduzida uma segunda revisão de segurança adversarial completa (não só do diff recente, mas do projeto inteiro), com foco especial em três frentes: criptografia/merge do cofre, superfície de IPC do processo principal do Electron (incluindo a janela oculta do CloudKit), e autenticação/storage do mobile (incluindo os módulos nativos Swift). A criptografia, o fluxo OAuth PKCE do Android e os módulos nativos iOS não apresentaram achados exploráveis. Dois achados de severidade alta e um de severidade média foram identificados e tratados:

9.1 Exclusão de conta sem autenticação real no desktop (Alto)

`account:delete` (apaga o cofre local e o remoto no Drive/iCloud, ação irreversível) só pedia confirmação por Touch ID quando disponível (`canPromptDeviceAuth()` , só verdadeiro no macOS com Touch ID) — no Windows, Linux, ou Mac sem Touch ID, a ação seguia sem nenhuma verificação real no processo principal, com a única barreira sendo um `window.confirm()` cosmético no renderer, pulável por qualquer script injetado por uma dependência comprometida. **Correção:** a senha de acesso agora é reverificada (`verifyLocalAccount`) no processo principal antes de qualquer exclusão, incondicionalmente, em todas as plataformas — o Touch ID (quando disponível) continua sendo pedido depois, como segunda camada. A interface (`VaultScreen.jsx`) trocou o alerta de confirmação por um formulário real pedindo a senha.

9.2 Janela principal do desktop sem restrição de navegação (Alto)

A janela principal do Electron (a única com acesso à ponte `window.secpass` completa) não tinha `will-navigate` nem `setWindowOpenHandler` registrados, ao contrário da janela oculta do CloudKit, que já tinha essa proteção corretamente (seção 2.1). Um renderer comprometido podia navegar a própria janela pra uma página remota (`window.location = "..."`) — o `preload.js` roda de novo no documento novo e reexpõe a ponte lá, fora da política de segurança de conteúdo do app (`default-src 'self'`), livre pra exfiltrar o cofre decifrado. **Correção:** como essa SPA nunca precisa de navegação de verdade (as telas trocam via estado do React), toda navegação da janela principal agora é bloqueada incondicionalmente.

9.3 Bloqueio de login burlável via relógio do sistema (Médio — risco aceito)

O bloqueio progressivo após tentativas de senha erradas compara o tempo de expiração contra `Date.now()` , o relógio do próprio aparelho. Quem tem acesso físico ao aparelho desbloqueado pode adiantar o relógio do sistema pra pular a espera, repetidamente. Diferente dos achados acima, isso **não tem uma correção real**: verificar a hora contra um servidor externo não resolve, porque o mesmo atacante só precisa desligar a rede antes de mexer no relógio, e o app teria que confiar no relógio local mesmo assim (senão travaria qualquer uso legítimo offline). Foi documentado como risco aceito diretamente no código (`src/utils/loginThrottle.js` , `desktop/src/main/core/loginThrottle.js`) e aqui: a defesa real contra força bruta é o custo computacional do PBKDF2 (600 mil iterações, seção 4.1), não esse contador da interface, que cobre só o caso comum de alguém tentando adivinhar de cabeça. Ver também 13.4.

10. Testes Automatizados

Suíte	Cobertura	Testes
Mobile (<code>__tests__</code> / , preset <code>jest-expo</code>)	Criptografia do cofre, storage/sync, merge, autenticação (incluindo o novo fluxo Android via navegador), throttle de login, geração de senha, componentes de UI, monitoramento de erros	210 testes / 17 arquivos
Desktop (<code>desktop/__tests__</code> / , Jest + Babel próprios)	Criptografia (parte do mobile), preferência de sync, storage/migração de backend, throttle e guarda de login (portados do mobile nesta fase)	64 testes / 6 arquivos

11. Ferramentas de Engenharia e Processo de Build

11.1 Plugins de configuração customizados do Expo

- `patch-package` — corrige bugs de aspas dentro de pacotes do `node_modules`, reaplicados automaticamente após todo `npm install` via um script `postinstall`.
- **Config plugin** `withSafeShellScriptPaths.js` — corrige o mesmo tipo de problema no projeto Xcode gerado a cada `expo prebuild`.
- **Config plugin** `withReleaseSigning.js` — injeta a assinatura de release real (keystore local, fora de versionamento) no `android/app/build.gradle` gerado, evitando que builds de release caíssem no keystore de debug público do template Expo.
- **Config plugin** `withAndroidGoogleOAuthRedirect.js` — registra o intent-filter do esquema de URL usado no retorno do OAuth do Google no Android (ver seção 6.1.1).
- **Config plugin** `withCloudKitCapability.js` — injeta as entitlements de iCloud/CloudKit no app iOS gerado. **Atualmente desativado** (removido da lista de plugins) porque exige um perfil de provisionamento com capacidade iCloud, indisponível sem uma conta paga do Apple Developer Program — builds atuais usam assinatura "Apple Development" gratuita, que não suporta essa entitlement.

11.2 Perfis de build (EAS)

O arquivo `eas.json` define perfis de build gerenciados pelo Expo Application Services para Android e iOS. Na prática, nesta fase do projeto, builds locais (`expo run:android` / `expo run:ios`, com `--configuration Release` para produção) foram preferidos aos builds em nuvem do EAS por serem dezenas de vezes mais rápidos (minutos contra mais de uma hora de fila).

11.3 Empacotamento do desktop (electron-builder)

O app desktop é empacotado com `electron-builder`, com alvos distintos para cada sistema operacional:

Comando	Uso	Saída
<code>npm run dist:dmg</code>	Build universal de produção para macOS	<code>.dmg</code> único rodando nativamente em Intel e Apple Silicon (<code>--universal</code>)
<code>npm run dist:release</code>	Build de macOS com assinatura completa	<code>.dmg</code> / <code>.zip</code> com Developer ID + notarização (requer certificado Apple pago, ver 12.2)
<code>npm run dist:win</code>	Build de produção para Windows	Instalador NSIS (<code>SecPass Setup x.x.x.exe</code>), arquitetura x64

Bug encontrado e corrigido nesta fase: a configuração original do alvo Windows (`"win": { "target": ["nsis"] }`) não especificava arquitetura explicitamente. Ao compilar num Mac com

Apple Silicon (arm64), o electron-builder assumiu por padrão a arquitetura do host, gerando um instalador **arm64** — incompatível com a esmagadora maioria dos PCs Windows, que são x64. A configuração foi corrigida para fixar `"arch": ["x64"]` explicitamente, garantindo compatibilidade nativa com o parque de máquinas Windows dominante (x64 também roda, via emulação, em Windows on ARM).

O executável de ambos os sistemas usa ícones de marca próprios do app (`build/icon.icns` no macOS, `build/icon.ico` no Windows), em vez do ícone genérico do Electron.

12. Estado de Prontidão para Produção e Comercialização

12.1 O que já está pronto

- Criptografia E2E completa e revisada adversarialmente (11 correções de segurança aplicadas)
- Sincronização funcional via Google Drive e iCloud, com escolha manual e migração
- Log de auditoria à prova de adulteração
- Suítes de teste automatizado no mobile (210) e no desktop (64), todas passando
- Mecanismos de build reproduzíveis (patches + plugins) para o ambiente específico deste projeto
- Estrutura de assinatura/notarização do desktop pronta para receber um certificado real
- **Cliente desktop Windows funcional** (instalador NSIS x64), a partir do mesmo código-fonte do macOS
- **Autenticação do Google Drive no Android reconstruída** com fluxo OAuth via navegador, eliminando a falha do SDK nativo (`AutoManageHelper`) que ocorria de forma consistente em builds sideloaded
- **Ícone de marca próprio** em todos os executáveis desktop (macOS e Windows), substituindo o ícone genérico do Electron
- **Atualização manual** (pull-to-refresh no mobile, botão "Atualizar" no desktop) complementando o pull automático no desbloqueio
- **Teste real de sincronização cruzada validado manualmente**: builds de produção (Release, minificados) instalados e testados em Android físico, emulador Android, iPhone físico e macOS, confirmando que uma credencial criada em um dispositivo aparece corretamente nos demais

12.2 O que falta (bloqueadores técnicos)

- **Extensão de AutoFill no iOS** (preencher senhas em outros apps/Safari) — deliberadamente adiada para um projeto separado, ainda não iniciada.
- **Assinatura/notarização real do desktop** — a estrutura existe (seção 11.3), mas nunca rodou de ponta a ponta com um certificado Apple Developer ID real (sem conta paga disponível no ambiente atual). O mesmo vale para assinatura de código Authenticode no Windows — o instalador atual não é assinado, o que pode acionar avisos do SmartScreen.
- **Dependências do desktop com vulnerabilidades conhecidas** — `electron` / `electron-builder` / `extract-zip` / `node-tar` têm alertas de segurança no `npm audit`, que exigiriam uma atualização com potencial quebra de compatibilidade — decisão ainda não tomada com o usuário.

- **iCloud/CloudKit em produção real** — a capability foi desativada no build atual do iOS (ver 11.1) por falta de perfil de provisionamento pago; o backend Google Drive é o único validado ponta a ponta nesta fase.

12.3 O que falta (fora do código — jurídico/negócio)

- Revisão jurídica da política de privacidade e termos de serviço para o mercado-alvo
- Definição de modelo de negócio (gratuito, assinatura, etc.) e infraestrutura de cobrança, se aplicável
- Processo de submissão às lojas (App Store/Play Store) e/ou distribuição direta (Microsoft Store ou instalador avulso) para Windows — contas de desenvolvedor, revisão de conteúdo, capturas de tela, descrição
- Suporte ao usuário e canal de reporte de vulnerabilidades de segurança

13. Limitações Conhecidas e Decisões de Design

13.1 Confiança no primeiro sync ("trust on first sync")

Como descrito na seção 6.4, um dispositivo completamente novo não tem como verificar criptograficamente, por si só, que o cofre remoto que está baixando é genuíno e atual — essa é uma limitação estrutural de qualquer sincronização E2E sem um servidor que arbitra ordem/autenticidade. Resolver isso de forma definitiva exigiria adicionar infraestrutura de servidor própria, o que está fora do escopo atual do projeto.

13.2 Ambiente de desenvolvimento específico deste projeto

Diversas correções de build (patches, plugins, flags de assinatura) existem especificamente porque este projeto é desenvolvido dentro de uma pasta sincronizada pelo Google Drive. Se o projeto for movido para fora dessa pasta no futuro, essas correções deixam de ser necessárias (mas continuam inofensivas caso permaneçam).

13.3 Builds locais completos e validados

Diferente de uma fase anterior deste projeto, builds locais completos (incluindo `pod install` /Xcode para iOS e Gradle para Android, em modo Release) foram executados e validados de ponta a ponta nesta fase, em dispositivos físicos reais, sem bloqueios de ferramental.

13.4 Bloqueio de login sem servidor (ver 9.3)

O contador de tentativas de senha erradas depende do relógio do próprio aparelho, sem nenhum árbitro externo. Isso é um limite estrutural inerente a um app sem servidor (a mesma categoria de limite descrita em 13.1 pra sincronização) — quem tem acesso físico ao aparelho desbloqueado pode adiantar o relógio do sistema pra pular a espera. Não há uma correção completa possível sem adicionar um servidor próprio (fora do escopo do projeto) — a defesa real contra força bruta continua sendo o custo computacional do PBKDF2 (seção 4.1).